

Scenario A — Basic Mixed Workload

Input (Quantum = 3, Lower # = Higher Priority):

Process	Arrival	Burst	Priority
P1	0	8	2
P2	1	4	1
P3	2	9	3
P4	3	5	2
P5	4	2	4

Results:

Metric	Round Robin	Priority
Avg WT	14.60	10.00
Avg TAT	20.20	15.60
Avg RT	4.60	9.20

Conclusion: Priority won on WT and TAT. Round Robin won on RT. Starvation risk detected in Priority — P3 had WT = 22 which is far above average.

Analysis: Priority Scheduling won on average WT and TAT because it served high-priority processes first. Round Robin won on Response Time (4.60 vs 9.20) because every process received CPU access quickly in rotation. Starvation risk was detected in Priority Scheduling P3 had a waiting time of 22, significantly above the average of 10.00.

Q1: Which algorithm gave better average waiting time?

Priority — 10.00 vs 14.60.

Q2: Which algorithm gave better response time?

Round Robin — 4.60 vs 9.20

Q3: Did high-priority processes gain advantage?

Yes — they completed with below-average waiting time.

Q4: Did Round Robin appear more balanced?

Yes — all processes received CPU time in rotation.

Q5: Was starvation observed?

Yes — max WT (22) far exceeds average (10.00).

Scenario B — Urgency Case

Input (Quantum = 4, Lower # = Higher Priority):

Process	Arrival	Burst	Priority
P1	0	12	1
P2	0	3	4
P3	2	5	2
P4	3	2	3

Results:

Metric	Round Robin	Priority
Avg WT	8.25	10.75
Avg TAT	13.75	16.25
Avg RT	4.25	10.75

Conclusion: Round Robin won on all 3 metrics. Priority served P1 (high priority, long burst) immediately and made P2 (short, low priority) wait — showing the urgency trade-off. RR interleaved all processes fairly.

Analysis: Round Robin won on all three metrics. Priority Scheduling kept running P1 (highest priority, burst = 12) without interruption, causing P2 (short burst = 3, low priority) to wait even though it could have finished quickly. Round Robin interleaved all processes fairly and produced better overall results. This demonstrates that Priority Scheduling is not always the most efficient algorithm — urgency comes at a cost to short low-priority processes.

Q1: Better average waiting time?

Round Robin — 8.25 vs 10.75.

Q2: Better response time?

Round Robin — 4.25 vs 10.75.

Q3: Did high-priority processes gain advantage?

Yes — P1 was served immediately under Priority.

Q4: Did Round Robin appear more balanced?

Yes — all processes were interleaved fairly.

Q5: Was starvation observed?

No significant starvation, but P2 waited much longer than necessary under Priority.

Scenario C — Fairness Case

Input (Quantum = 2, Lower # = Higher Priority):

Process	Arrival	Burst	Priority
P1	0	6	1
P2	0	6	1
P3	0	6	1
P4	1	4	1

Results:

Metric	Round Robin	Priority
Avg WT	13.25	8.75
Avg TAT	18.75	14.25
Avg RT	2.75	8.75

Conclusion: All processes have the same priority, so Priority Scheduling degenerates into FCFS — it serves them in arrival order with no preemption. Round Robin gave a much better response time (2.75 vs 8.75) because every process got CPU access quickly in rotation.

Analysis: All processes share the same priority value (1), so Priority Scheduling behaves exactly like FCFS — it runs each process to completion in arrival order with no preemption. Round Robin produced a significantly better response time (2.75 vs 8.75) because every process received CPU access within the first quantum cycle. This confirms Round Robin's fairness advantage when all processes are equally important.

Q1: Better average waiting time?

Priority (as FCFS) — 8.75 vs 13.25.

Q2: Better response time?

Round Robin — 2.75 vs 8.75.

Q3: Did high-priority processes gain advantage?

No — all priorities were equal, so no differentiation occurred.

Q4: Did Round Robin appear more balanced?

Yes — every process got CPU time quickly regardless of order.

Q5: Was starvation observed?

No — equal priorities prevent starvation in both algorithms here.

Scenario D — Starvation Risk

Input (Quantum = 3, Lower # = Higher Priority):

Process	Arrival	Burst	Priority
P1	0	3	1
P2	0	3	1
P3	1	3	1
P4	2	3	1
P5	0	10	5

Results:

Metric	Round Robin	Priority
Avg WT	6.60	5.40
Avg TAT	11.00	9.80
Avg RT	5.40	5.40

Conclusion: P5 has the lowest priority — Priority Scheduling kept preempting it until all other processes finished, giving it WT = 12. Round Robin served P5 fairly in rotation regardless of its priority. This demonstrates starvation risk in Priority Scheduling.

Analysis: P5 has the lowest priority (5) while all other processes have priority 1. Under Priority Scheduling, P5 could not run until P1, P2, P3, and P4 all finished — giving it a waiting time of 12, while all others had waiting times of 0 to 3. This is a clear case of starvation. Round Robin served P5 every 3 time units in rotation regardless of its priority, treating it fairly. In a real system, if new high-priority processes kept arriving, P5 could wait indefinitely under Priority Scheduling.

Q1: Better average waiting time?

Priority — 5.40 vs 6.60.

Q2: Better response time?

Equal — both 5.40.

Q3: Did high-priority processes gain advantage?

Yes — they all finished before P5 even got CPU time under Priority.

Q4: Did Round Robin appear more balanced?

Yes — P5 was served in rotation every quantum.

Q5: Was starvation observed?

Yes — P5 WT = 12 vs average 5.40 under Priority Scheduling.

Scenario E — Validation Test

Invalid inputs used:

- Quantum = -1
- P1: Arrival = -2
- P2: Burst = -3
- P3: Priority = 0

Field	Value Entered	Why It Is Invalid
Time Quantum	-1	Must be ≥ 1
P1 Arrival Time	-2	Cannot be negative
P2 Burst Time	-3	Must be ≥ 1
P3 Priority	0	Must be ≥ 1

Expected result: Simulator rejects all inputs and displays error messages without running.

Actual result: Simulator displayed 4 error messages and refused to simulate — validation working correctly.

Formulas Used:

Metric	Formula
Turnaround Time (TAT)	Finish Time – Arrival Time
Waiting Time (WT)	TAT – Burst Time
Response Time (RT)	First CPU access time – Arrival Time
Average	Sum of all values ÷ Number of processes